Weak Binary Semaphore Mutual Exclusion in TLA+

Final Project Report for CE356

Tianbin Wang Kehao Zhang

June 2026

Abstract

We study the CE356 project on weak binary semaphores. We focus on n-process mutual exclusion. We use only weak binary semaphores. We use TLA+ and TLC to check correctness. We compare a simple baseline with a Udding-style protocol. We use Peterson's algorithm as background. We then explain why weak semaphores create a fairness problem. We run TLC experiments for $n = 2$ through $n = 7$. We also add fairness assumptions and temporal properties to the Udding model. We then check no-starvation for small instances. Our results show that both models satisfy the checked safety properties. Our results also show that the Udding model supports fairness-oriented liveness checking at small scale. We argue that this extra power comes with a much larger state space.

1 Introduction

The project guideline asks us to implement n-process mutual exclusion with only weak binary semaphores. The guideline also asks us to show correctness in TLA+. This task creates a direct tension between exclusion and fairness. A very small semaphore protocol can satisfy exclusion. A fairness-oriented protocol needs much more structure.

We follow the same direction as our midterm report. We now extend that direction into a complete final report. We compare two designs:

- a simple baseline that uses a single weak binary semaphore to protect the critical section;
- a Udding-style protocol that introduces a staged admission discipline to better control overtaking.

We use this comparison to answer three questions. First, what can a naive weak-semaphore mutex already guarantee? Second, why is that guarantee still not enough for fairness? Third, how much extra modeling cost do we pay when we move to a fairness-oriented design?

2 Background

2.1 Weak binary semaphores and fairness

A binary semaphore takes values in $\{0, 1\}$. A binary semaphore supports the usual P and V operations. In the weak case, the semaphore does not promise a strongly fair service rule for blocked processes. As a result, one waiting process may keep losing contention even when the whole system still makes progress.

This issue makes the project interesting. We do not only want mutual exclusion. We also want to understand how much structure a protocol needs when the primitive itself does not provide enough fairness.

2.2 Classical mutual exclusion background: Peterson's algorithm

Peterson's algorithm is a classical software-only mutual exclusion algorithm. We use it as conceptual background. In the two-process case, each process raises a flag and writes the shared variable `turn`:

```
flag[i] := TRUE;
turn := j;
while flag[j] /\ turn = j do skip;
CS;
flag[i] := FALSE;
```

Peterson's algorithm is useful because it shows how shared variables can enforce mutual exclusion. However, Peterson's algorithm is not the final protocol for this project. The project requires weak binary semaphores. We therefore use Peterson's algorithm as background, not as the main experimental baseline.

2.3 Semaphore baseline used in our experiments

For the actual weak-semaphore baseline, we use the simplest protocol:

```
NCS;
P(mutex);
CS;
V(mutex);
```

This baseline matches the project restriction. This baseline also shows the fairness problem very clearly. The protocol enforces exclusion, but the protocol does not stop indefinite overtaking.

3 Why Udding's Algorithm Matters

Udding's algorithm is directly relevant to our project. Udding designed this algorithm to address individual starvation under weak semaphores. The algorithm does not let every process race directly for the final mutex. The algorithm uses a staged pipeline. The pipeline separates outer contenders from processes that are already closer to the critical section.

The protocol we model has the following structure:

```
P(enter); ne := ne + 1; V(enter);
P(queue); P(enter); nm := nm + 1; ne := ne - 1;
if ne > 0 then V(enter) else V(mutex);
V(queue);
P(mutex); nm := nm - 1;
CS;
if nm > 0 then V(mutex) else V(enter)
```

The role of each shared object is as follows:

- `enter` controls whether new outer contenders may continue inward;
- `queue` serializes competition around the second `P(enter)`;
- `mutex` controls the innermost path to the critical section;
- `ne` counts outer-layer contenders;
- `nm` counts processes waiting in the inner layer.

This design matters for fairness. The extra structure is not cosmetic. The structure prevents later arrivals from interfering arbitrarily with processes that are already deeper in the waiting pipeline. In other words, Udding already contains this fairness-oriented idea. Our TLA+ additions only make that idea explicit enough for TLC to check.

4 TLA+ Models

4.1 Baseline model

We model the baseline in TLA+ with one semaphore variable and one per-process control-state function. Each process moves through `ncs`, `trying`, `cs`, and `exit`. We check the following safety property:

```
InCS == {i \in Proc : pc[i] = "cs"}
MutualExclusion == Cardinality(InCS) <= 1
```

This model stays simple on purpose. We want this model to represent the smallest semaphore-based protocol that still solves mutual exclusion.

4.2 Udding model

We model the Udding protocol in TLA+ with the following variables:

- `pc`
- `enter`
- `queue`
- `mutex`
- `ne`
- `nm`

The model breaks the protocol into small atomic actions. The model includes the outer entry phase, the queueing phase, the handoff phase, the mutex-wait phase, and the exit phase. In addition to `TypeOK` and `MutualExclusion`, we also check:

```
SplitBinary == enter + mutex \in {0, 1}
```

This invariant captures the intended split-binary-semaphore discipline between `enter` and `mutex`.

4.3 Fairness and liveness layer

To move beyond pure safety, we add the following definitions to the Udding model:

- a per-process request predicate `Requesting(i)`;
- a per-process critical-section predicate `InCSProc(i)`;
- a temporal property `StarvationFree(i) == Requesting(i) ~> InCSProc(i)`;
- a system-level property `NoStarvation`.

We also add two fairness layers:

- `ProcessFairness`, which applies weak fairness to each process step;
- `SemaphoreFairness`, which applies strong fairness to the key blocking acquisitions `PEnter1`, `PQueue`, `PEnter2`, and `PMutex`.

The resulting specification is:

```
FairSpec == Spec /\ ProcessFairness /\ SemaphoreFairness
```

This refinement matters. If we only use coarse process-level fairness, TLC finds a starvation counterexample. If we add fairness assumptions at the semaphore-acquisition level, TLC can check the no-starvation property for small instances.

4.4 Code excerpts

The baseline and Udding specifications are short enough to inspect directly. We include the full TLA+ code in the appendices. We do this to keep the report self-contained.

5 Experimental Method

We run TLC on both models with the following process-set parameter:

$$Proc = \{1, 2, \ldots, n\}, \quad n \in \{2, 3, 4, 5, 6, 7\}.$$

For the baseline model, we check:

- `TypeOK`
- `MutualExclusion`

For the Udding model, we check:

- `TypeOK`
- `MutualExclusion`
- `SplitBinary`

For fairness-oriented liveness checking, we also prepare:

- a two-process liveness configuration using `FairSpec`;
- a three-process liveness configuration using `FairSpec`;
- the temporal property `NoStarvation`.

We save the raw TLC output for every run. We then summarize all runs in one CSV table. We use that table for both the report tables and the plots.

6 Experimental Results

6.1 Complete result table

Table 1: TLC results for $n = 2$ through $n = 7$

n	Baseline Gen.	Baseline Dist.	Baseline Depth	Udding Gen.	Udding Dist.	Udding Depth
2	21	12	5	131	88	23
3	73	32	6	973	520	33
4	225	80	7	6401	2880	43
5	641	192	8	38961	15264	53
6	1729	448	9	224257	78208	63
7	4481	1024	10	1236929	390016	73

6.2 Growth plots

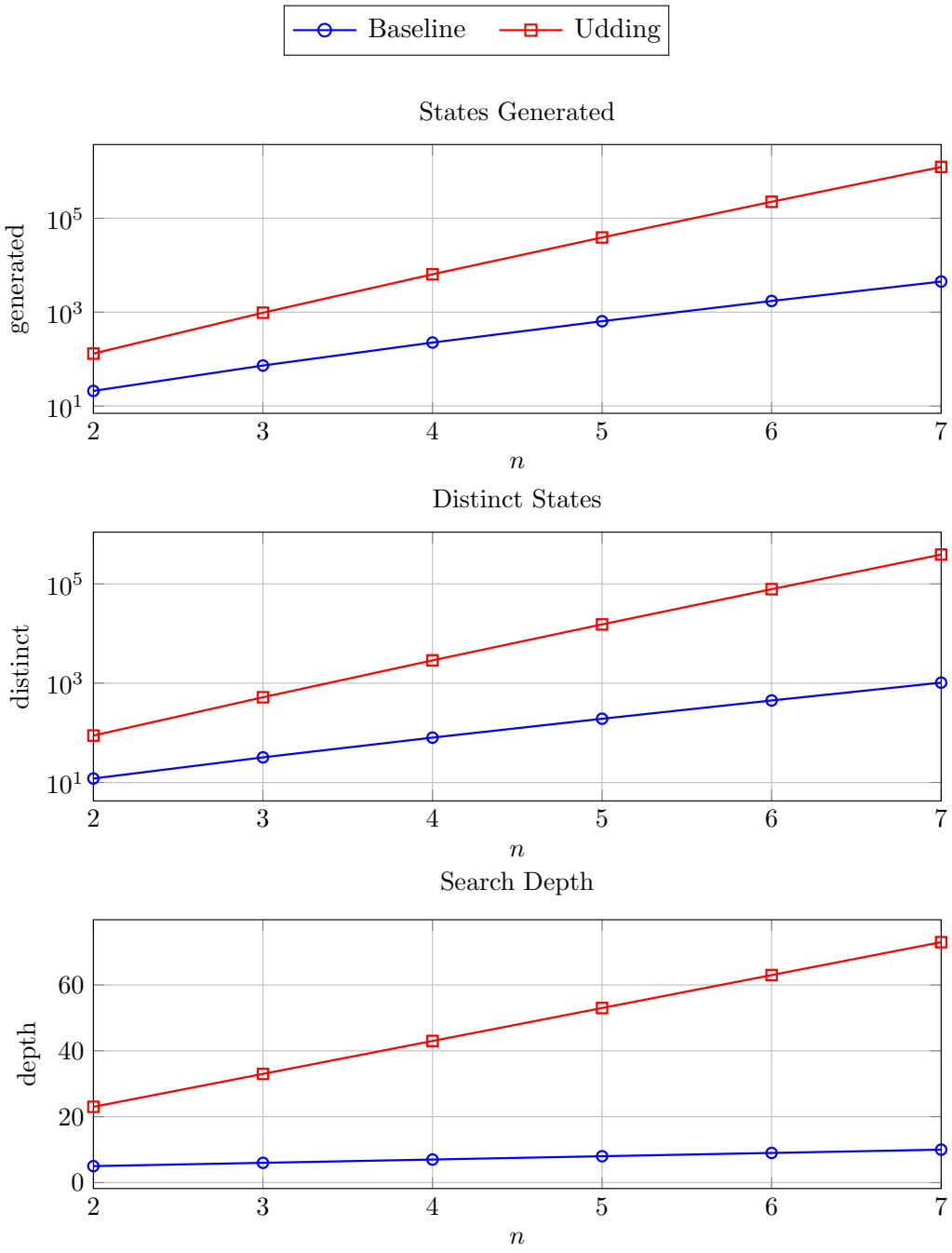

Figure 1: Growth of generated states, distinct states, and search depth for the baseline and Udding models.

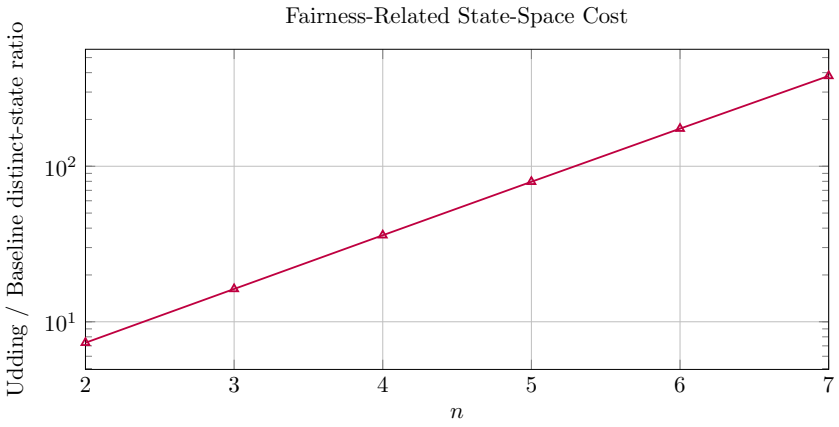

Figure 2: Distinct-state ratio of Udding to baseline. This ratio captures how much extra state-space cost we pay for fairness-related protocol structure.

6.3 Fairness-oriented liveness results for Udding

After enriching the Udding model with `FairSpec` and `NoStarvation`, we ran TLC on the following small parameter instances:

Table 2: Liveness-oriented TLC results for Udding

n	Specification	Property	Result	Time
2	`FairSpec`	`NoStarvation`	pass	under 1 second
3	`FairSpec`	`NoStarvation`	pass	about 11 seconds

These results matter for the main claim of the project. The results show that the TLA+ model captures the safety discipline of Udding's protocol. The results also show that the model captures its intended anti-starvation behavior under explicit fairness assumptions.

7 Comparison and Analysis

7.1 Both models satisfy the checked safety properties

For all tested values of n from 2 through 7, both models pass TLC with no invariant violation. This result shows that both protocols satisfy mutual exclusion in the parameter range that we explored.

7.2 The real difference is fairness-related structure

The baseline is enough if we only ask for exclusion. The baseline is small, direct, and easy to verify. However, the baseline does not regulate how contenders are admitted. Under weak fairness, this means a waiting process can still be overtaken again and again.

The Udding protocol is different in a fundamental way. The protocol adds a staged admission structure. The protocol separates outer waiting, queue serialization, and inner waiting. This structure makes the model fairness-conscious. The larger state space is therefore not accidental overhead. The larger state space reflects extra commitments about how the protocol organizes contention.

This interpretation is now supported in two ways:

- by safety-oriented parameter sweeps for $n = 2$ through $n = 7$;
- by liveness-oriented `NoStarvation` checks for $n = 2$ and $n = 3$ under explicit fairness assumptions.

7.3 State-space cost of fairness

The clearest quantitative result is the ratio between the distinct-state counts of the Udding and baseline models:

Table 3: Distinct-state ratio of Udding to baseline

n	Baseline Distinct	Udding Distinct	Ratio
2	12	88	7.33x
3	32	520	16.25x
4	80	2880	36.00x
5	192	15264	79.50x
6	448	78208	174.57x
7	1024	390016	380.88x

This trend supports our main interpretation. As we increase the number of processes, the fairness-aware admission discipline of Udding becomes much more expensive to model and verify than the simple baseline.

7.4 Peterson, baseline, and Udding in perspective

We can now place the three ideas in a useful hierarchy:

- Peterson's algorithm gives us classical mutual exclusion intuition.
- The single-`mutex` semaphore baseline gives us the correct weak-semaphore comparison point.
- Udding's algorithm gives us a fairness-oriented semaphore design for the actual project setting.

This layered presentation is useful because it shows our path clearly. We start from general mutual exclusion background. We then move to the specific fairness problem caused by weak binary semaphores.

8 Discussion of Fairness

Fairness is the main theme of this project. We do not claim that the weak semaphore itself gives us fairness for free. Instead, we make the following points:

- the baseline model shows how exclusion can be achieved with minimal structure;
- the Udding model shows how much additional structure is needed when we care about overtaking and fairness;
- under explicit fairness assumptions on the key semaphore-acquisition actions, TLC confirms `NoStarvation` for the small instances we checked;
- the experimental state-space gap makes the cost of that fairness-oriented structure concrete.

This is why fairness should stay at the center of the report. If we only reported mutual exclusion, the Udding model would look unnecessarily complicated. Once we make fairness explicit, the extra semaphores and counters become easier to understand and easier to justify.

9 Conclusion

We addressed the CE356 weak-binary-semaphore project by combining background theory, TLA+ modeling, and parameterized TLC experiments. We used Peterson's algorithm to motivate classical mutual exclusion. We used a single weak-semaphore mutex as the true baseline because it matches the project constraint. We then modeled a Udding-style protocol to capture a more fairness-conscious admission discipline.

Our experiments for $n = 2$ through $n = 7$ show that both models satisfy the checked safety properties. Our experiments also show that the Udding model incurs a much larger state-space cost. In addition, under explicit fairness assumptions at the semaphore-acquisition level, the Udding model satisfies the temporal no-starvation property for the small instances that we checked. We interpret this result as the price of fairness-related coordination under weak binary semaphore assumptions.

Our final conclusion is simple:

> we can obtain mutual exclusion cheaply, but we cannot obtain fairness cheaply.

That is exactly why Udding's algorithm is a meaningful subject for formal specification and verification in this project.

A Baseline TLA+ Code

```
------------------------------ MODULE BaselineMutex ------------------------------
EXTENDS Naturals, FiniteSets

CONSTANT Proc

ASSUME Proc /= {}

VARIABLES pc, mutex

Vars == << pc, mutex >>

Init ==
    /\ pc = [i \in Proc |-> "ncs"]
    /\ mutex = 1

EnterTrying(i) ==
    /\ pc[i] = "ncs"
    /\ pc' = [pc EXCEPT ![i] = "trying"]
    /\ mutex' = mutex

AcquireMutex(i) ==
    /\ pc[i] = "trying"
    /\ mutex = 1
    /\ pc' = [pc EXCEPT ![i] = "cs"]
    /\ mutex' = 0

LeaveCS(i) ==
    /\ pc[i] = "cs"
    /\ pc' = [pc EXCEPT ![i] = "exit"]
    /\ mutex' = mutex

ReleaseMutex(i) ==
    /\ pc[i] = "exit"
    /\ mutex = 0
    /\ pc' = [pc EXCEPT ![i] = "ncs"]
    /\ mutex' = 1

ProcStep(i) ==
    EnterTrying(i)
    \/ AcquireMutex(i)
    \/ LeaveCS(i)
    \/ ReleaseMutex(i)

Next ==
    \E i \in Proc : ProcStep(i)
```

```
TypeOK ==
    /\ pc \in [Proc -> {"ncs", "trying", "cs", "exit"}]
    /\ mutex \in {0, 1}

InCS ==
    {i \in Proc : pc[i] = "cs"}

MutualExclusion ==
    Cardinality(InCS) <= 1

Spec ==
    Init /\ [][Next]_Vars

=============================================================================
```

B Udding TLA+ Code

```
------------------------------ MODULE UddingMutex ------------------------------
EXTENDS Naturals, FiniteSets

CONSTANT Proc

ASSUME Proc /= {}

VARIABLES pc, enter, queue, mutex, ne, nm

Vars == << pc, enter, queue, mutex, ne, nm >>

Init ==
    /\ pc = [i \in Proc |-> "ncs"]
    /\ enter = 1
    /\ queue = 1
    /\ mutex = 0
    /\ ne = 0
    /\ nm = 0

StartAttempt(i) ==
    /\ pc[i] = "ncs"
    /\ pc' = [pc EXCEPT ![i] = "e1"]
    /\ UNCHANGED << enter, queue, mutex, ne, nm >>

PEnter1(i) ==
    /\ pc[i] = "e1"
    /\ enter = 1
    /\ pc' = [pc EXCEPT ![i] = "ne_inc"]
    /\ enter' = 0
```

```
    /\ UNCHANGED << queue, mutex, ne, nm >>

IncNE(i) ==
    /\ pc[i] = "ne_inc"
    /\ pc' = [pc EXCEPT ![i] = "e1_rel"]
    /\ ne' = ne + 1
    /\ UNCHANGED << enter, queue, mutex, nm >>

VEnter1(i) ==
    /\ pc[i] = "e1_rel"
    /\ enter = 0
    /\ pc' = [pc EXCEPT ![i] = "q_wait"]
    /\ enter' = 1
    /\ UNCHANGED << queue, mutex, ne, nm >>

PQueue(i) ==
    /\ pc[i] = "q_wait"
    /\ queue = 1
    /\ pc' = [pc EXCEPT ![i] = "e2_wait"]
    /\ queue' = 0
    /\ UNCHANGED << enter, mutex, ne, nm >>

PEnter2(i) ==
    /\ pc[i] = "e2_wait"
    /\ enter = 1
    /\ pc' = [pc EXCEPT ![i] = "nm_inc"]
    /\ enter' = 0
    /\ UNCHANGED << queue, mutex, ne, nm >>

IncNM(i) ==
    /\ pc[i] = "nm_inc"
    /\ pc' = [pc EXCEPT ![i] = "ne_dec"]
    /\ nm' = nm + 1
    /\ UNCHANGED << enter, queue, mutex, ne >>

DecNE(i) ==
    /\ pc[i] = "ne_dec"
    /\ ne > 0
    /\ pc' = [pc EXCEPT ![i] = "handoff"]
    /\ ne' = ne - 1
    /\ UNCHANGED << enter, queue, mutex, nm >>

InnerHandoff(i) ==
    /\ pc[i] = "handoff"
    /\ pc' = [pc EXCEPT ![i] = "q_rel"]
    /\ IF ne > 0
          THEN /\ enter' = 1
               /\ mutex' = mutex
```

```
         ELSE /\ mutex' = 1
              /\ enter' = enter
    /\ UNCHANGED << queue, ne, nm >>

VQueue(i) ==
    /\ pc[i] = "q_rel"
    /\ queue = 0
    /\ pc' = [pc EXCEPT ![i] = "mu_wait"]
    /\ queue' = 1
    /\ UNCHANGED << enter, mutex, ne, nm >>

PMutex(i) ==
    /\ pc[i] = "mu_wait"
    /\ mutex = 1
    /\ pc' = [pc EXCEPT ![i] = "nm_dec"]
    /\ mutex' = 0
    /\ UNCHANGED << enter, queue, ne, nm >>

DecNM(i) ==
    /\ pc[i] = "nm_dec"
    /\ nm > 0
    /\ pc' = [pc EXCEPT ![i] = "cs"]
    /\ nm' = nm - 1
    /\ UNCHANGED << enter, queue, mutex, ne >>

LeaveCS(i) ==
    /\ pc[i] = "cs"
    /\ pc' = [pc EXCEPT ![i] = "ncs"]
    /\ IF nm > 0
         THEN /\ mutex' = 1
              /\ enter' = enter
         ELSE /\ enter' = 1
              /\ mutex' = mutex
    /\ UNCHANGED << queue, ne, nm >>

ProcStep(i) ==
      StartAttempt(i)
    \/ PEnter1(i)
    \/ IncNE(i)
    \/ VEnter1(i)
    \/ PQueue(i)
    \/ PEnter2(i)
    \/ IncNM(i)
    \/ DecNE(i)
    \/ InnerHandoff(i)
    \/ VQueue(i)
    \/ PMutex(i)
    \/ DecNM(i)
```

```
    \/ LeaveCS(i)

Next ==
    \E i \in Proc : ProcStep(i)

TypeOK ==
    /\ pc \in [Proc -> {"ncs", "e1", "ne_inc", "e1_rel", "q_wait", "e2_wait",
                       "nm_inc", "ne_dec", "handoff", "q_rel",
                       "mu_wait", "nm_dec", "cs"}]
    /\ enter \in {0, 1}
    /\ queue \in {0, 1}
    /\ mutex \in {0, 1}
    /\ ne \in Nat
    /\ nm \in Nat

SplitBinary ==
    enter + mutex \in {0, 1}

InCS ==
    {i \in Proc : pc[i] = "cs"}

MutualExclusion ==
    Cardinality(InCS) <= 1

InCSProc(i) ==
    pc[i] = "cs"

Requesting(i) ==
    pc[i] \in {"e1", "ne_inc", "e1_rel", "q_wait", "e2_wait",
               "nm_inc", "ne_dec", "handoff", "q_rel",
               "mu_wait", "nm_dec"}

StarvationFree(i) ==
    Requesting(i) ~> InCSProc(i)

NoStarvation ==
    \A i \in Proc : StarvationFree(i)

ProcessFairness ==
    \A i \in Proc : WF_Vars(ProcStep(i))

SemaphoreFairness ==
    \A i \in Proc :
        /\ SF_Vars(PEnter1(i))
        /\ SF_Vars(PQueue(i))
        /\ SF_Vars(PEnter2(i))
        /\ SF_Vars(PMutex(i))
```

```
Spec ==
    Init /\ [][Next]_Vars

FairSpec ==
    Spec /\ ProcessFairness /\ SemaphoreFairness

=============================================================================
```